

FIRE: Force-Informed Residual Policy for Contact-Rich Manipulation with Vision-Language-Action Models

Abstract—Vision-Language-Action (VLA) models adapt to manipulation tasks from a few demonstrations, but remain brittle in contact-rich assembly, where visual observations fail to capture contact state and chunked VLA actions cannot react at the control rate. We present Force-Informed Residual Policy (FIRE), which augments a frozen, fine-tuned VLA with a force-aware residual RL policy. The VLA provides long-horizon action chunks, while the residual policy runs at every control step, observes force, and adds bounded corrections through a slow-fast asynchronous loop. Trained in simulation with only a sparse task-success reward and a task-agnostic force-aware dense reward, FIRE transfers zero-shot to a torque-controlled manipulator in the real world. On three real-robot assembly tasks, Peg Insert, Gear Mesh, and Nut Thread, it improves success over the VLA-based policy by an average of 64.5 percentage points (up to 90.0), matching or exceeding an RL policy trained with task-specific dense rewards while requiring no such reward engineering. Moreover, the same residual policy applies to any VLA backbone, transferring across multiple models without architectural changes and improving each backbone on all tasks. Experimental videos and parameters are available on the project page: <https://anonymous.4open.science/w/FIRE-3E69/>.

Index Terms—Reinforcement Learning, Force and Tactile Sensing, Compliant Assembly, Vision-Language-Action.

I. INTRODUCTION

Recent advances in robot learning have been substantially accelerated by the emergence of large-scale vision-language-action (VLA) models, a class of foundation models in robotics. Trained on internet-scale visual and linguistic data alongside diverse robot embodiments, VLA models demonstrate strong generalization capabilities and semantic understanding across a broad range of manipulation tasks [1]–[7]. A key advantage of VLA models is their capacity to adapt to novel tasks and environments from only a small number of demonstrations, establishing their potential as general-purpose agents capable of interpreting semantic instructions and planning goal-directed behaviors [4], [5].

Despite these advances, VLA models face fundamental limitations in contact-rich manipulation tasks that demand precise physical interaction [8], [9], even as they achieve strong performance on tasks that do not require precise contact regulation, such as pick-and-place. Fig. 1 shows the three representative contact-rich assembly tasks considered in this study, namely Peg Insert, Gear Mesh, and Nut Thread. These limitations stem from three factors. First, VLA models rely predominantly on visual and linguistic inputs, leaving force and tactile signals, modalities critical for physical interaction, insufficiently utilized during both training and inference [10]–[12]. Moreover, VLA models generate actions in chunks that are executed open-loop, and their inference latency exceeds the control period. Consequently, even with

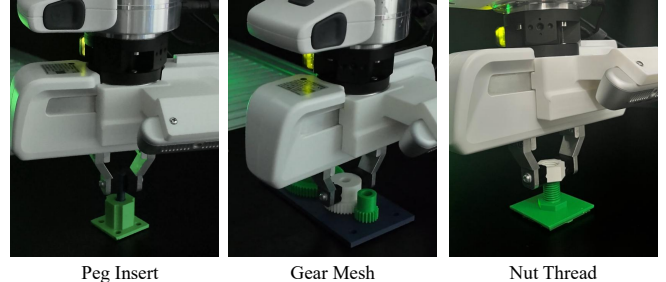


Fig. 1. Real-robot execution of the three contact-rich assembly tasks. The tasks require precise contact regulation beyond visual observation alone.

force sensing they cannot react to contact events at the control rate [13], [14]. Second, visual occlusion caused by the end-effector (EEF) or manipulated objects during precision manipulation leads to the loss of task-critical state information [10]. Third, the collection of high-quality demonstrations for contact-rich tasks incurs prohibitive cost, as teleoperation under contact uncertainty requires specialized hardware to convey contact forces to the operator [15], [16].

Reinforcement learning (RL)-based approaches, by contrast, can leverage force feedback for reactive contact control but are limited by high sample complexity in continuous state-action spaces and the need for carefully engineered task-specific dense reward functions [17], [18]. That is, these are per-step rewards that must be hand-designed anew for each task to encode its progress toward completion. Neither VLA nor RL alone is sufficient to address the demands of contact-intensive manipulation.

In this study, we propose Force-Informed Residual Policy (FIRE), an approach that combines a frozen VLA as the base policy with a residual policy trained via RL for precise contact control. FIRE addresses the three limitations above by leveraging the long-horizon planning capability of VLA models together with the force-aware reactive control capability of RL. Specifically, after fine-tuning only the action head of the VLA, the frozen model generates action chunks from visual and language inputs to provide a base trajectory, while the residual policy observes force sensor readings and outputs corrective actions. The final control command is the sum of the two outputs.

This residual learning structure enables the two approaches to compensate for each other’s weaknesses. Using the VLA output as the base trajectory warm-starts the RL agent, which then explores from a region near the goal. Conversely, the residual policy applies real-time corrective actions via force feedback, compensating for the VLA’s limited contact understanding and susceptibility to visual occlusion.

The primary contributions of this study are as follows. First, we propose FIRE, which combines a frozen VLA as the

base policy with a force-aware residual policy dedicated to contact control. The two policies use separated observations to specialize into distinct roles (long-horizon planning and closed-loop contact control) while their actions are aligned in a common relative-pose representation and combined by simple addition. This delegates force sensing to the residual policy for contact control. Second, building on this combination, we cast contact correction as a residual RL problem trained without any task-specific dense reward. The goal-directed guidance normally supplied by such rewards comes instead from the VLA-based action, which warm-starts exploration near the goal. The only additional reward is a task-agnostic, force-aware dense reward. It shapes how contact unfolds without altering the optimal policy. Third, our design allows the same residual policy to be ported across different VLA backbones without architectural changes. Since only the common action representation must match, a single residual policy applies to GR00T, $\pi_{0.5}$, and OpenVLA as-is.

The remainder of this study is organized as follows. Section II reviews related work, Section III details the FIRE formulation, Section IV presents the simulation and real-robot experiments, and Section V concludes the study.

II. RELATED WORK

A. Vision-Language-Action Models

VLA models represent an approach that integrates large-scale visual-language pretraining with robot control to learn general-purpose manipulation policies. Following RT-1 [1], which pioneered policy learning from large-scale real-world data using a Transformer-based architecture, RT-2 [2] substantially improved zero-shot generalization by jointly fine-tuning on internet-scale visual-language data. This line of work was extended in Open X-Embodiment [3], which unified cross-embodiment transfer learning by aggregating data from 22 robot platforms. OpenVLA [4] further improved the accessibility of semantic grounding as an open-source model. More recently, $\pi_{0.5}$ [19] demonstrated efficient fine-tuning from only a small number of demonstrations through flow-based action generation, while RDT-1B [6] explored diffusion-based action generation and GR00T N1 [7] extended the VLA paradigm to humanoid robots, collectively broadening the scope of VLA-based approaches.

Despite these advances, current VLA models share a common limitation in that they typically rely on vision, language, and proprioceptive robot state (e.g., joint positions and EEf pose), while contact-related signals such as force, friction, and slip, which require dedicated force or tactile sensing, are not directly utilized [1]–[7], [10], [11].

B. Force and Tactile Integration for Contact-Rich Manipulation

To address the absence of contact sensing in VLA models, considerable effort has been devoted to integrating force or tactile modalities into policy learning. RDP [13] introduced a slow-fast hierarchical structure that combines a low-frequency diffusion policy with high-frequency tactile

feedback, improving responsiveness and stability in contact-rich manipulation. ForceVLA [10] proposed an end-to-end approach that dynamically integrates 6-axis force/torque (F/T) signals with visual-language embeddings through a mixture-of-experts fusion module. TA-VLA [11] systematically explored the design space for incorporating torque signals into VLA architectures, demonstrating that inserting torque adapters into the decoder and predicting torque as an auxiliary output improves performance on contact-rich tasks.

These works experimentally demonstrate the importance of force and tactile modalities for contact-rich manipulation. However, they share a common approach of modifying the VLA architecture or retraining the full model weights to internalize the new modality, which incurs substantial training cost, a cost that must be paid anew for each new backbone as VLA models continue to advance. FIRE takes a different approach by fine-tuning only the action head of the VLA and then freezing the entire model, while delegating force sensing exclusively to a separate residual policy. The formulation thus carries over to new backbones that produce compatible action chunks, without architectural modification or full-model retraining.

C. Residual Policy Learning

Residual policy learning is an approach that augments an existing controller or pretrained policy by adding corrective actions learned via RL, without modifying the base policy itself. Johannink et al. [20] applied residual RL on top of a hand-designed feedback controller, with the residual policy observing the EEf pose and F/T measurements, to solve block assembly tasks involving contact and friction on a real robot. Their approach demonstrated higher sample efficiency and noise robustness compared to training RL from scratch. Schoettler et al. [17] extended residual learning to industrial insertion tasks, showing that, because the base controller already provides a coarse solution, real-world training becomes feasible with sparse rewards alone.

More recently, the base policy in residual learning has shifted from hand-designed controllers to learning-based policies. Ankile et al. [14] applied residual RL on top of a frozen diffusion policy, proposing a structure in which the action chunk-based behavior cloning policy functions as a trajectory planner providing long-horizon guidance, while the residual policy functions as a closed-loop controller performing precise corrections. This approach achieved substantially improved success rates on insertion tasks with sub-millimeter clearance using only sparse rewards, though without incorporating force feedback into the residual policy. This direction has recently been extended to large-scale VLA-based policies. PLD [21] freezes a VLA generalist and trains lightweight residual RL specialists to probe its failure regions, then distills the resulting rollouts back into the generalist via supervised fine-tuning. PLD, however, employs the residual policy as a vision-based data-generation tool for offline self-improvement. Prior work thus leaves unaddressed the combination of a frozen large-scale VLA-based policy with a force-aware residual policy. FIRE combines these

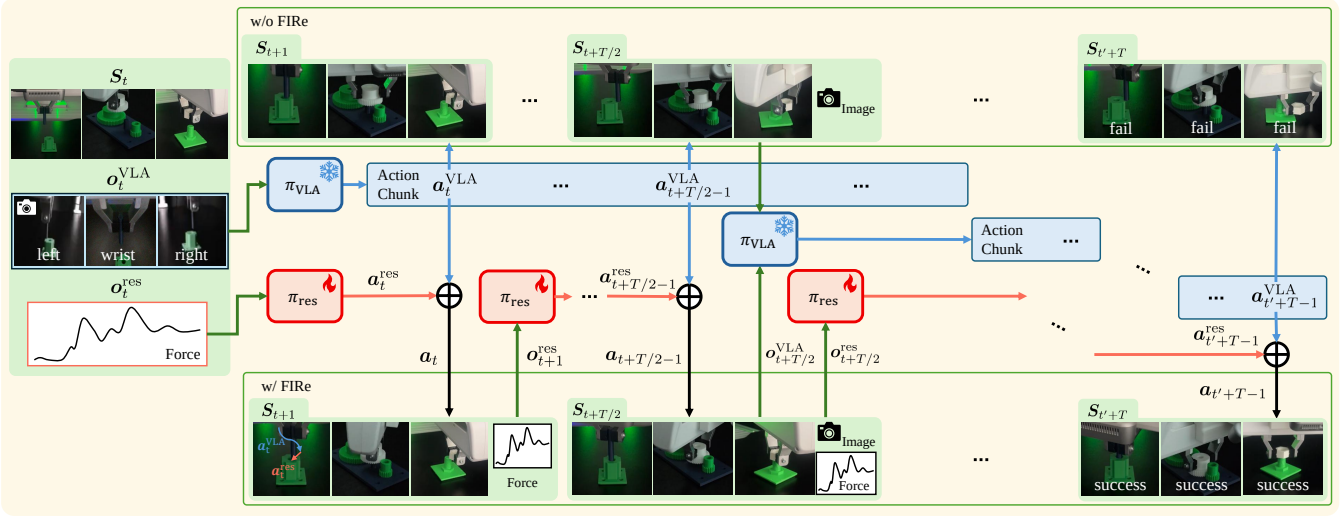


Fig. 2. Overview of FIRE. A frozen VLA generates action chunks from its own VLA observation, which includes visual and language inputs, while a force-aware residual policy runs at the control rate and adds bounded corrections from its own observation, which includes proprioceptive state and force.

two components by incorporating force sensing exclusively into the residual policy, enabling force-aware contact control without modifying the VLA architecture.

III. METHODOLOGY

A. System Overview

FIRE generates the final control command by adding the corrective output of a residual policy to the base action of a frozen VLA, as illustrated in Fig. 2. The VLA generates a base action chunk of length T from visual and language inputs, functioning as a trajectory planner that provides long-horizon guidance. Because each chunk is executed to completion, one action per control step, the VLA is queried once every T control steps. The residual policy π_{res} observes proprioceptive state and force sensor readings at every control step and outputs a corrective action, functioning as a closed-loop controller that performs real-time contact corrections.

This slow-fast structure allows the residual policy to compensate for physical disturbances that arise while the VLA executes an action chunk in open-loop. The approach keeps the VLA weights frozen without modification and is therefore compatible in principle with any VLA that produces action chunks. In this study we use GR00T as our primary backbone and further validate the approach on $\pi_{0.5}$ and OpenVLA.

B. Problem Formulation

This study models the residual policy learning problem as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \rho, \rho_0, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\rho(s' | s, a)$ is the transition dynamics, ρ_0 is the initial-state distribution, r is the reward function, and $\gamma \in (0, 1]$ is the discount factor. The agent does not have direct access to the state $s_t \in \mathcal{S}$, which is instantiated below as the privileged simulator state, and instead learns a policy π_{res} that maximizes the expected cumulative reward using only the partial observation o_t^{res} .

Residual Observation. The residual observation o_t^{res} accessible at deployment consists of the EEF pose $p_{ee}^{rel} \in \mathbb{R}^7$ (position and quaternion) expressed in the frame of the fixed object (e.g., the hole), EEF velocity $v_{ee} \in \mathbb{R}^6$, contact force $F_t \in \mathbb{R}^3$, and the previous step action a_{t-1} (the final command executed at step $t-1$), collected as

$$o_t^{res} = \{p_{ee}^{rel}, v_{ee}, F_t, a_{t-1}\}. \quad (1)$$

Privileged State. The MDP state $s_t \in \mathcal{S}$ is instantiated as the privileged state, additional information that can only be obtained from the simulator. In addition to all elements of o_t^{res} , s_t includes joint positions q_j , the absolute and fixed-object-relative poses of the held object p_h and p_h^{rel} , the absolute pose of the fixed object p_f , the proportional gain k_p of the low-level controller, the pose threshold ϵ_p , and the reference contact force F_{ref} used in the force-aware reward, collected as

$$s_t = \{o_t^{res}, q_j, p_h, p_h^{rel}, p_f, k_p, \epsilon_p, F_{ref}\}. \quad (2)$$

All absolute poses are expressed in the robot base frame.

Action Space. The VLA-based action is a *relative pose*, a displacement of the EEF from its current pose that consists of a 3-DOF position offset and a 3-DOF rotation offset in Euler angles, both expressed in the current EEF frame. The action of the residual policy is a bounded correction $a_t^{res} \in \mathcal{A} = \mathbb{R}^6$ added to the base action. Each component is clipped to $[a_{min}, a_{max}]$, chosen small relative to the base action, so the residual refines the VLA trajectory rather than overriding it. The final control command, the sum of the two, is also a relative pose.

VLA Observation. The frozen base policy is not part of the residual MDP. From the perspective of the residual policy it belongs to the environment, and it receives its own observation o_t^{VLA} , comprising RGB images $\{I_k\}$ from multiple cameras (two external cameras and one wrist camera in our setup, detailed in Sec. IV-A), the absolute EEF pose $p_{ee}^{abs} \in \mathbb{R}^7$ (position and quaternion), and a natural-language

Algorithm 1 FIRE deployment control loop

Require: frozen VLA π_{VLA} , residual policy π_{res} , chunk length T

```
1: observe  $\mathbf{o}^{\text{VLA}}$ ;  $\mathbf{A}^{\text{VLA}} \leftarrow \pi_{\text{VLA}}(\mathbf{o}^{\text{VLA}})$ ;  $i \leftarrow 0$ 
   (initial blocking chunk)
2: while episode not terminated do
3:   observe  $\mathbf{o}^{\text{res}}$ 
4:   if  $i = \lfloor T/2 \rfloor$  then
5:     observe  $\mathbf{o}^{\text{VLA}}$ ;  $\text{fut} \leftarrow \text{ASYNCINFER}(\pi_{\text{VLA}}, \mathbf{o}^{\text{VLA}})$  (prefetch next chunk, non-blocking)
6:   end if
7:   if  $i = T$  then
8:      $\mathbf{A}^{\text{VLA}} \leftarrow \text{AWAIT}(\text{fut})$ ;  $i \leftarrow 0$  (swap in prefetched chunk)
9:   end if
10:   $\mathbf{a}^{\text{res}} \leftarrow \pi_{\text{res}}(\mathbf{o}^{\text{res}})$  (every control step)
11:   $\mathbf{a} \leftarrow \mathbf{a}_i^{\text{VLA}} + \mathbf{a}^{\text{res}}$  (combined relative-pose command)
12:  execute  $\mathbf{a}$  with the task-space impedance controller
13:   $i \leftarrow i + 1$ 
14: end while
```

task instruction ℓ , collected as

$$\mathbf{o}_t^{\text{VLA}} = \{ \{ \mathbf{I}_k \}, \mathbf{p}_{\text{ee}}^{\text{abs}}, \ell \}. \quad (3)$$

Unlike the residual observation $\mathbf{o}_t^{\text{res}}$, $\mathbf{o}_t^{\text{VLA}}$ is expressed in absolute coordinates, carries visual and semantic context but no contact force, and is read only once per action chunk.

C. Force-Informed Residual Policy

The residual policy π_{res} takes the observation $\mathbf{o}_t^{\text{res}}$ as input and outputs a corrective action that is combined with the base action of the frozen VLA to form the final control command. At every T control steps, the VLA generates an action chunk $\mathbf{A}^{\text{VLA}} = (\mathbf{a}_t^{\text{VLA}}, \dots, \mathbf{a}_{t+T-1}^{\text{VLA}})$, where t denotes the step at which the chunk was generated. At each control step t , the residual policy outputs a corrective action $\mathbf{a}_t^{\text{res}}$, which is added to the t -th element $\mathbf{a}_t^{\text{VLA}}$ of the most recently generated chunk. The base action and the residual correction share the same relative-pose representation (Sec. III-B), so the final command is formed by direct addition,

$$\mathbf{a}_t = \mathbf{a}_t^{\text{VLA}} + \mathbf{a}_t^{\text{res}}. \quad (4)$$

The combined command $\mathbf{a}_t$ is executed on the robot by a task-space impedance controller [22].

FIRE executes the combination of base and residual actions as a slow-fast loop, summarized in Algorithm 1. The residual policy π_{res} runs at every control step, whereas the VLA is queried only once per chunk of length T . Because a VLA inference takes longer than one control step, FIRE prefetches the next chunk asynchronously at the midpoint of the current one and swaps it in once the current chunk is exhausted, so the control loop runs at a fixed rate without stalling.

The residual policy is trained via PPO [23] with an asymmetric actor-critic structure, in which the actor uses the deployment-accessible observation $\mathbf{o}_t^{\text{res}}$ while the critic

additionally leverages the privileged state $\mathbf{s}_t$ available only in simulation. The policy is trained entirely in simulation and deployed zero-shot on the real robot without any real-world fine-tuning. FIRE uses a sparse task-success reward r_s and a task-agnostic force-aware dense reward r_f , combined as

$$r = r_s + r_f, \quad r_f = r_p + r_{sh}, \quad (5)$$

which we refer to as the FIRE reward (FIRE-rew). The task-success reward is the binary indicator $r_s = 1[\text{success}]$. The force-aware reward consists of a direct penalty r_p on the magnitude of force-limit violations and a potential-based shaping term r_{sh} that encourages violation reduction. Both terms are defined with respect to a reference contact-force level F_{ref} at which penalization begins.

Although r_f is dense, being defined at every control step, it is task-agnostic: the same functional form and hyperparameters are shared across all tasks. The direct penalty

$$r_p = -\alpha \cdot \max(0, \|\mathbf{F}_{t+1}\| - F_{\text{ref}}), \quad (6)$$

with scale $\alpha > 0$, penalizes the instantaneous exceedance of F_{ref} and thereby bounds the contact force. The shaping term

$$r_{sh} = \lambda_f (\gamma \phi_f(\mathbf{s}_{t+1}) - \phi_f(\mathbf{s}_t)), \quad (7)$$

with scale $\lambda_f > 0$, is potential-based with potential

$$\phi_f(\mathbf{s}) = -\log(c_f \cdot \max(0, \|\mathbf{F}(\mathbf{s})\| - F_{\text{ref}}) + 1), \quad (8)$$

where $\mathbf{F}(\mathbf{s})$ denotes the contact force in state $\mathbf{s}$ and $c_f > 0$ controls the sensitivity of the potential to exceedances of F_{ref} . The potential is zero when the contact force is at or below F_{ref} and becomes increasingly negative as the exceedance grows, so r_{sh} is positive on any transition that decreases the exceedance. Being potential-based, it densifies the learning signal during over-force contact without altering the optimal policy [24].

Note that r_f alone provides no incentive to make or maintain contact, since both the penalty r_p and the potential ϕ_f vanish at any contact force at or below F_{ref} . The drive toward contact comes instead from the task-success reward r_s and the VLA-based action, which carries the policy into the contact region that completion requires. Within this goal-directed drive, r_f only shapes how contact unfolds, favoring task completion with bounded, decreasing contact forces.

IV. EXPERIMENTS

We evaluate FIRE with five research questions, presented in the order in which the experiments appear. (RQ1) Without any task-specific dense reward, does FIRE reach success rates comparable to an RL policy trained with such rewards in simulation? (RQ2) Under an identical training budget, does the VLA warm-start enable FIRE to learn where from-scratch RL trained with the same FIRE-rew fails or converges slowly? (RQ3) On a real robot, does FIRE achieve higher success on contact-rich assembly than the VLA-based policy and an RL policy trained with task-specific dense rewards? (RQ4) Do the force observation and the force-aware reward in the residual policy each contribute to real-world success? (RQ5) Can the residual formulation be applied across different action-chunking VLA backbones?

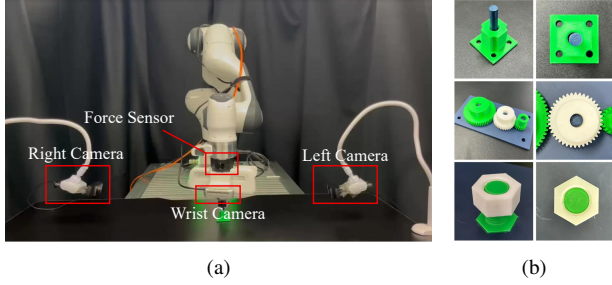


Fig. 3. Experimental setup. (a) The real-robot platform is a Franka FR3 with a wrist-mounted 3-axis force sensor and three cameras. (b) The three assembly tasks are Peg Insert, Gear Mesh, and Nut Thread, which use 3D-printed parts.

A. Experimental Setup

Tasks. We consider the three contact-rich manipulation tasks Peg Insert, Gear Mesh, and Nut Thread (Fig. 3). Peg Insert requires aligning and inserting a cylindrical peg into a tight-clearance hole. Gear Mesh requires aligning two gear teeth so that the gears engage and rotate. Nut Thread requires aligning a nut with a bolt and threading it to a target depth. A trial is successful when the final lateral error and insertion state fall within task-specific tolerances (part dimensions and per-task tolerances are listed in Table I). For Nut Thread, following [25], success is determined by manual inspection because thread engagement is not observable from the end-effector pose. These tasks are sensitive to small pose errors, visual occlusion, and contact-state uncertainty.

Robot and force sensing. All experiments use a Franka FR3 with a wrist-mounted 3-axis force sensor that provides the contact force $\mathbf{F}_t$; the same hardware is used across all baselines, ablations, and FIRE.

Simulation and sim-to-real. Policies are trained in the Isaac Lab simulator [26] and deployed zero-shot on the real robot. Following FORGE [25], we bridge the sim-to-real gap with domain randomization and observation noise on the pose and force inputs, applied identically to every method. As in FORGE, the fixed-object pose used to compute $\mathbf{p}_{ee}^{rel}$ is assumed known from the workcell setup, with its placement error absorbed into the pose observation noise.

Low-level controller and action. The base policy outputs relative poses and the residual policy outputs bounded corrections in the same representation (Sec. III-B), executed by a task-space impedance controller. The residual policy and the task-space impedance controller run at different rates (Table I); each combined command is interpolated to the controller rate between successive policy steps.

Training. All RL policies are trained with PPO using an asymmetric actor-critic and identical hyperparameters across methods. The VLA regenerates an action chunk every T control steps, and the force-aware reward uses fixed scales α , λ_f , and c_f together with a reference contact force F_{ref} that is randomized during training. The remaining PPO hyperparameters are available on the project page. All evaluations use, for each method and task, the checkpoint that recorded the highest success rate during training.

Baselines and ablations. We compare FIRE against base-

TABLE I

IMPLEMENTATION SETTINGS, SHARED ACROSS TASKS AND METHODS.

Setting	Value
<i>Task and success criteria</i>	
Peg diameter	8 mm
Peg-hole radial clearance	0.08 mm
Success tol., Peg Insert	lateral ≤ 5 mm, depth ≤ 25 mm
Success tol., Gear Mesh	lateral ≤ 10 mm, below surface
Success tol., Nut Thread	nut not liftable (manual)
<i>Low-level control and action</i>	
Residual action $\mathbf{a}_t^{res}$ (bounded)	$\in \mathbb{R}^6$
VLA action, typical magnitude	5 cm
Residual action bound a_{max}, a_{min}	± 2.5 cm ($0.5 \times$ VLA)
Residual policy rate	15 Hz
Impedance controller rate	1 kHz
VLA action-chunk length T	16 control steps
<i>FIRE-rew hyperparameters (5)</i>	
Force-penalty scale α	1×10^{-5}
Shaping scale λ_f	1×10^{-5}
Potential coefficient c_f	1.0
Reference contact force F_{ref}	10 N (eval; rand. in training)
<i>Training (PPO, asymmetric actor-critic)</i>	
Parallel environments	64
Training epochs	200
Checkpoint selection	best success rate

lines and two ablations. VLA executes the action-head fine-tuned VLA without residual correction, isolating the frozen base policy. RL dense is an RL policy trained from scratch without a VLA-based action using a task-specific dense reward. We adopt FORGE [25] as its representative, a strong contact-rich manipulation baseline. It requires a reward engineered separately for each task, in our case three distinct dense-reward functions for the three tasks, whereas FIRE trains with the identical FIRE-rew (5) on all tasks. RL FIRE-rew trains the same from-scratch PPO controller with FIRE-rew, isolating the effect of the VLA warm-start under a matched reward. FIRE w/o force removes the contact-force entries $\mathbf{F}_t$ from both the residual observation and the reward. FIRE obs-only keeps $\mathbf{F}_t$ in the observation but removes the force-aware reward r_f , and FIRE uses force in both. All methods share the same evaluation protocol per task.

B. Simulation Performance and Sample Efficiency (RQ1, RQ2)

We first evaluated, in simulation, whether FIRE reaches success rates comparable to RL dense without any task-specific dense reward. FIRE reached peak success rates of 100% on Peg Insert, 100% on Gear Mesh, and 97% on Nut Thread (Fig. 4), comparable to RL dense (94%, 100%, and 100%, respectively). This indicated that the VLA-based policy together with a force-aware residual was sufficient to reach dense-reward-level performance in simulation while eliminating task-specific reward engineering, addressing RQ1.

We then addressed RQ2 by comparing all methods under an identical training budget, so that the curves are read against training epochs (Fig. 4). The comparison isolates the effect of the VLA warm-start under a matched reward

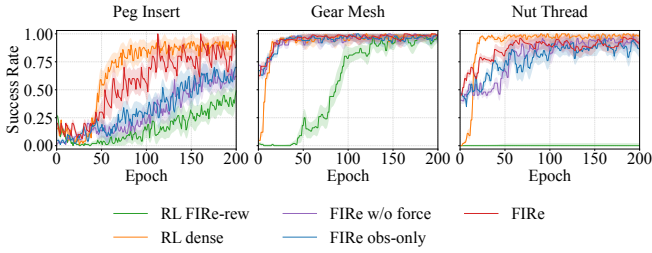


Fig. 4. Simulation learning curves under a matched budget across methods. Shaded bands indicate variation across three seeds.

TABLE II

REAL-ROBOT TASK SUCCESS RATES OVER 30 TRIALS PER METHOD. THE INC. ROW IS FIRE’S GAIN OVER THE VLA-BASED POLICY.

Method	Peg Insert	Gear Mesh	Nut Thread
VLA	23.3% (7/30)	10.0% (3/30)	3.3% (1/30)
RL dense	86.7% (26/30)	100.0% (30/30)	33.3% (10/30)
FIRE w/o force	43.3% (13/30)	93.3% (28/30)	6.7% (2/30)
FIRE obs-only	70.0% (21/30)	100.0% (30/30)	16.7% (5/30)
FIRE (ours)	96.7% (29/30)	100.0% (30/30)	33.3% (10/30)
Inc.	+73.4	+90.0	+30.0

(FIRE vs. RL FIRE-rew). To compare learning speed on a common scale, we report the epoch at which each method first reaches an 80% success reference, a fixed high bar that all converging methods clear. On Peg Insert, RL FIRE-rew failed to learn the task within the budget, plateauing around 40% and never reaching 50%, whereas FIRE, warm-started near the goal by the VLA-based policy, reached the reference around epoch 107. On Gear Mesh both eventually converged, but FIRE reached it by epoch ~ 15 versus epoch ~ 101 for RL FIRE-rew. On Nut Thread, RL FIRE-rew remained at 0% success throughout training, whereas FIRE reached the reference around epoch 37. The VLA warm-start thus made residual learning feasible where RL FIRE-rew failed or converged slowly. Since VLA is not trained through online interaction, it is shown as a reference success level.

C. Real-World Success and Robustness (RQ3, RQ4)

We next evaluated transfer to the real robot, which constitutes our central evaluation. Table II reports real-robot success rates over 30 trials per method. Across the three tasks, FIRE improved over the VLA-based policy by 73.4, 90.0, and 30.0 percentage points (%p) on Peg Insert, Gear Mesh, and Nut Thread, respectively (Table II, Inc.). On Peg Insert, FIRE reached the highest success rate, 96.7%, well above the VLA-based policy (23.3%) and on par with RL dense (86.7%, within 10.0 %p). This answered RQ3. FIRE matched RL dense without any task-specific dense reward while substantially improving over the VLA-based policy. RL dense’s 86.7% closely matched the 84% reported for FORGE under comparable conditions [25], indicating that our real-robot setup is faithful. To verify that the low VLA success is not an artifact of fine-tuning, we evaluated the same action-head fine-tuned VLA on a pick-and-place task that does not require precise contact regulation, with the same demonstration budget. It attained 96.7% success,

indicating that the VLA failures are specific to contact-rich interaction. The same pattern held on Gear Mesh, where FIRE attained 100% while the VLA-based policy reached only 10.0%. Here FIRE, FIRE obs-only, and RL dense all saturated at 100%. Gear Mesh therefore corroborated RQ3 but did not separate the ablation, leaving the ablation evidence for RQ4 to rest on Peg Insert. Videos of FIRE executing the three tasks are available on the project page: <https://anonymous.4open.science/w/FIRE-3E69/>.

To assess how each use of contact force contributes, we compared the ablation variants on Peg Insert, reported in Table II. Adding the force observation (FIRE obs-only) raised success over the force-blind FIRE w/o force from 43.3% to 70.0% (+26.7 %p). Additionally introducing the force-aware reward (FIRE) raised it to 96.7% (a further +26.7 %p, i.e., +53.4 %p over the force-blind baseline). Both components thus contributed substantially, answering RQ4. The force observation and the force-aware reward were each necessary, and neither alone was sufficient.

Fig. 5 shows real-world robustness to initial XY pose perturbations of up to 16 mm. On Peg Insert, FIRE sustained near-perfect success across the full perturbation range, while RL dense degraded at large perturbations and the force-blind ablation failed across most of the range. A per-episode failure-mode breakdown (Fig. 6) explains how each use of force helps, using three geometric failure modes, namely drift-off (the tool drifts off and fails to stay on the hole), entry failure (the tool is aligned over the hole but does not enter), and surface miss (the tool stays in contact with the surface but fails to align with the hole). The force-blind FIRE w/o force failed predominantly by drift-off, in 11 of its 17 failures. Adding the force observation (FIRE obs-only) reduced drift-off failures from 11 to 3 by keeping the tool on or near the hole, though 4 entry failures and 2 surface misses remained; adding the force-aware reward (FIRE) then cleared the remaining failures, reducing the total from 9 to 1, leaving a single drift-off failure. Thus the force *observation* enabled the closed-loop alignment needed to locate the hole, while the force-aware *reward* removed the residual entry failures, each contributing to success. On Gear Mesh (Fig. 5), the VLA-based policy failed overwhelmingly by entry failure (23/30), whereas FIRE sustained 100% across the perturbation range.

On Nut Thread, the hardest task, each use of contact force still improved success monotonically, from 6.7% (FIRE w/o force) to 16.7% (obs-only) to 33.3% (full FIRE) (Table II), corroborating RQ4. Success nonetheless capped at 33.3%, and the reason lies in the dominant failure: about 60% of failures were aligned but not threaded, with lateral error indistinguishable from the successes (1.32 versus 1.38 mm, Mann–Whitney $p = 0.92$). The failure is therefore not one of positioning but of thread engagement, which is governed mainly by the axial reaction torque about the bolt [27] and is invisible to our three-axis force sensing. The geometric per-mode breakdown (drift-off, entry, surface miss) is thus uninformative for Nut Thread, so we present that analysis on Peg Insert, while Fig. 6 still reports all three tasks.

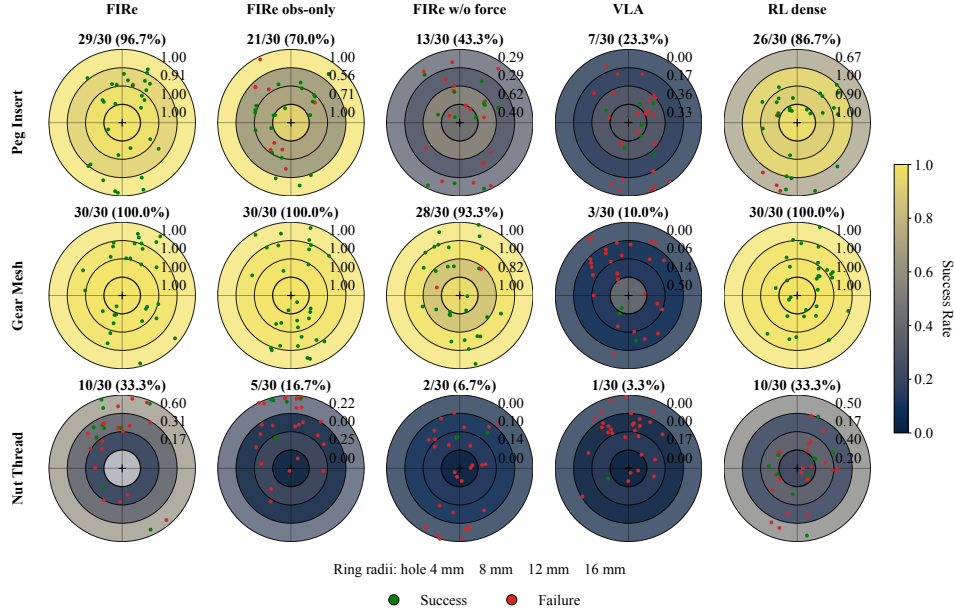


Fig. 5. Real-world robustness under initial XY pose perturbations (initial-noise centers aligned across methods), with rows for Peg Insert, Gear Mesh, and Nut Thread and columns for the five methods, where VLA is the GR00T base policy. Concentric rings mark initial XY perturbation radii of 4, 8, 12, and 16 mm from the true target position. On Peg Insert the innermost ring coincides with the hole itself. Per-ring success rates are annotated, and each panel reports the overall success rate. Green markers indicate successful trials and red markers indicate failures.

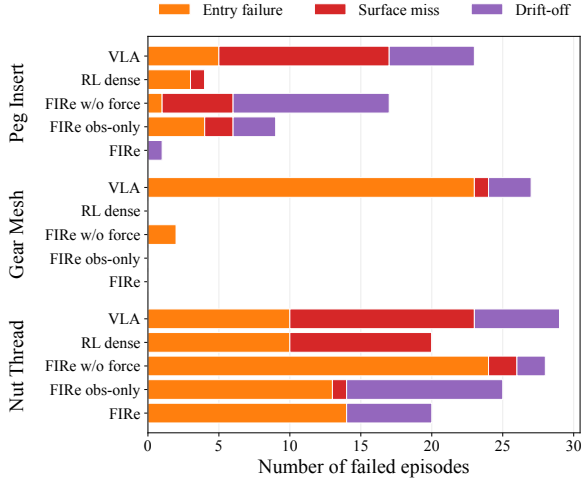


Fig. 6. Per-episode failure modes for the three tasks (real robot), with one bar per method stacked by geometric failure type.

TABLE III

VLA-ONLY AND VLA+FIRE SUCCESS RATE ACROSS BACKBONES AND TASKS. VALUES ARE REAL-ROBOT SUCCESS RATES.

	Peg Insert		Gear Mesh		Nut Thread	
Backbone	VLA	+FIRE	VLA	+FIRE	VLA	+FIRE
GR00T	23.3%	96.7%	10.0%	100.0%	3.3%	33.3%
$\pi_{0.5}$	20.0%	96.7%	66.7%	100.0%	0.0%	13.3%
OpenVLA	56.7%	90.0%	93.3%	100.0%	3.3%	16.7%

Even so, three-axis force alone already lifted every VLA backbone (Table III), validating the direction of delegating contact sensing to the residual policy; extending it to six-axis force/torque sensing is a natural path to closing the remaining gap on Nut Thread and similar torque-governed tasks, which

we leave to future work.

D. Generalizability Across VLA Backbones (RQ5)

To address RQ5, we evaluated GR00T, $\pi_{0.5}$, and OpenVLA under an identical protocol, each augmented with the same residual policy. The formulation plugs into any action-chunking backbone without architectural changes, requiring only compatible 6D EEF relative-pose action chunks so that the residual adds corrections in the same representation. Table III reports, for each backbone, the VLA-based policy and the same backbone with the FIRE residual, isolating the residual’s gain. The residual improved every backbone on every task. On Peg Insert and Gear Mesh it lifted all three backbones to at least 90.0%, from base rates as low as 10.0%. Nut Thread remained hardest, where the residual improved every backbone but success stayed at 33.3% or below. The magnitude of the gain thus varied across backbones and tasks, but the residual improved performance in every case.

V. CONCLUSION

In this study, we proposed FIRE, which augments a VLA-based policy with a force-aware residual policy dedicated to contact control, trained with only a sparse task-success reward and a task-agnostic force-aware dense reward. On three real-robot contact-rich assembly tasks, FIRE improved success over the VLA-based policy by 73.4, 90.0, and 30.0 percentage points, matched an RL policy trained with task-specific dense rewards, and was more robust to initial-pose perturbations than the baselines. A controlled ablation showed that the force observation and the force-aware reward each contributed, improving success monotonically on every task as each was added. In simulation as well, FIRE reached the success level of RL dense without any task-specific reward engineering, and under a matched training

budget the VLA warm-start enabled residual learning where from-scratch RL with the same reward failed or converged slowly. Finally, the same residual policy transferred across the GR00T, $\pi_{0.5}$, and OpenVLA backbones without architectural changes, improving every backbone on every task, although the size of the gain varied across backbones. Future work is to make a single FIRE training uniformly robust across diverse VLA backbones. A remaining limitation is the comparatively low success rate on Nut Thread, which we plan to address by extending the three-axis force sensor to six-axis force/torque and feeding the torque into both the residual observation and reward.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proceedings of Robotics: Science and Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2212.06817>
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning*, 2023, pp. 2165–2183. [Online]. Available: <https://arxiv.org/abs/2307.15818>
- [3] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar *et al.*, “Open X-embodiment: Robotic learning datasets and RT-X models,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2024, pp. 6892–6903. [Online]. Available: <https://arxiv.org/abs/2310.08864>
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the 8th Conference on Robot Learning*, 2024, pp. 2679–2713. [Online]. Available: <https://arxiv.org/abs/2406.09246>
- [5] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.24164>
- [6] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “RDT-1B: a diffusion foundation model for bimanual manipulation,” in *The Thirtieth International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.07864>
- [7] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” NVIDIA, Tech. Rep., 2025. [Online]. Available: <https://arxiv.org/abs/2503.14734>
- [8] H. Lee, S. Park, K. Jang, S. Kim, and J. Park, “Contact state estimation for peg-in-hole assembly using Gaussian mixture model,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 3349–3356, 2022.
- [9] H. Lee, E. Sung, S. You, and J. Park, “CC-STAR: An estimation for contact state transition using reconstruction-based anomaly detection for peg-in-hole assembly,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 15 486–15 492.
- [10] J. Yu, H. Liu, Q. Yu, J. Ren, C. Hao, H. Ding, G. Huang, G. Huang, Y. Song, P. Cai, W. Zhang, and C. Lu, “ForceVLA: Enhancing VLA models with a force-aware MoE for contact-rich manipulation,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=2845H8Ua5D>
- [11] Z. Zhang, H. Xu, Z. Yang, C. Yue, Z. Lin, H. ang Gao, Z. Wang, and H. Zhao, “TA-VLA: Elucidating the design space of torque-aware vision-language-action models,” in *Proceedings of the Conference on Robot Learning*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.07962>
- [12] J. Jones, O. Mees, C. Sferrazza, K. Stachowicz, P. Abbeel, and S. Levine, “Beyond sight: Finetuning generalist robot policies with heterogeneous sensors via language grounding,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.04693>
- [13] H. Xue, J. Ren, W. Chen, G. Zhang, Y. Fang, G. Gu, H. Xu, and C. Lu, “Reactive diffusion policy: Slow-fast visual-tactile policy learning for contact-rich manipulation,” in *Proceedings of Robotics: Science and Systems*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.02881>
- [14] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal, “From imitation to refinement — residual RL for precise assembly,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025, pp. 01–08. [Online]. Available: <https://arxiv.org/abs/2407.16677>
- [15] J. J. Liu, Y. Li, K. Shaw, T. Tao, R. Salakhutdinov, and D. Pathak, “FACTR: Force-attending curriculum training for contact-rich policy learning,” in *Proceedings of Robotics: Science and Systems*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.17432>
- [16] E. Sung, S. You, J. Shin, S. Park, S. Hur, J. Kim, S. Byun, C. Mun, H. Lee, D. Kim, and J. Park, “SNU-Avatar haptic arm: A hybrid QDD/DD operator interface for teleoperation,” *IEEE Access*, vol. 14, pp. 57 409–57 426, 2026.
- [17] G. Schoettler, A. Nair, J. Luo, S. Bahl, J. A. Ojea, E. Solowjow, and S. Levine, “Deep reinforcement learning for industrial insertion tasks with visual inputs and natural rewards,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2020, pp. 5548–5555. [Online]. Available: <https://arxiv.org/abs/1906.05841>
- [18] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, “SERL: A software suite for sample-efficient robotic reinforcement learning,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2024, pp. 16 961–16 969. [Online]. Available: <https://arxiv.org/abs/2401.16013>
- [19] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhahalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [20] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 International Conference on Robotics and Automation*, 2019, pp. 6023–6029. [Online]. Available: <https://arxiv.org/abs/1812.03201>
- [21] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu, “Self-improving vision-language-action models with data generation via residual RL,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://arxiv.org/abs/2511.00091>
- [22] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987.
- [23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [24] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [25] M. Noseworthy, B. Tang, B. Wen, A. Handa, C. Kessens, N. Roy, D. Fox, F. Ramos, Y. Narang, and I. Akinola, “FORGE: Force-guided exploration for robust contact-rich manipulation under uncertainty,” *IEEE Robotics and Automation Letters*, vol. 10, pp. 4436–4443, 2025. [Online]. Available: <https://arxiv.org/abs/2408.04587>
- [26] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlkar, B. Babich, G. State, M. Hutter, and A. Garg, “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023. [Online]. Available: <https://arxiv.org/abs/2301.04195>
- [27] T. Matsuno, J. Huang, and T. Fukuda, “Fault detection algorithm for external thread fastening by robotic manipulator using linear support vector machine classifier,” in *2013 IEEE International Conference on Robotics and Automation*. IEEE, 2013, pp. 3443–3450.